

Figure 1 gives the output of both compile (dynamic) and run-time memory allocation.

While executing, the compile memory allocation program results in a segmentation fault but the runtime or dynamic memory allocation program results in the character 'P' without errors.

To understand the above, we need to explore the memory layout of the C Program running process (Figure 2).

C program memory layout

The text segment contains the machine code of the compiled program.

Usually, the text segment is shareable so that only a single copy needs to be in memory for frequently executed programs, such as text editors, the C compiler, the shells, and so on. The text segment of an executable object file is often a read-only segment, which prevents a program from being accidentally modified.

The initialised data stores all global, static, constant, and external variables (declared with the external keyword) that are initialised beforehand. The data segment is not read-only, since the values of the variables can be altered at runtime. This segment can be further classified into an initialised read-only area and an initialised read-write area.

Data in this segment is initialised to arithmetic 0 before the program starts executing. Uninitialised data starts at the end of the data segment, and contains all global variables and static variables that are initialised to 0 or do not have explicit initialisation in source code.

Heap is the segment where *dynamic memory allocation* usually takes place. When some more memory needs to be allocated using the *malloc* and *calloc* functions, heap grows upward. The heap area is shared by all shared libraries and dynamically loaded modules in a process.

The stack segment is used to store all local variables and is used for passing arguments to the functions along with the return address of the

```
cm@cm-Vostro-3559:~/osfydec2k19$ vim compilealloc.c
cm@cm-Vostro-3559:~/osfydec2k19$ vim runtimealloc.c
cm@cm-Vostro-3559:~/osfydec2k19$ gcc compilealloc.c
cm@cm-Vostro-3559:~/osfydec2k19$ ./a.out
String OpenSourceForYou
Strlen 16
Segmentation fault (core dumped)
cm@cm-Vostro-3559:~/osfydec2k19$ gcc runtimealloc.c
cm@cm-Vostro-3559:~/osfydec2k19$ ./a.out
Strng OpenSourceForYou
Strlen 16
Char after pointer expression P
cm@cm-Vostro-3559:~/osfydec2k19$
```

Figure 1: Output of both compile (dynamic) and run-time memory allocation

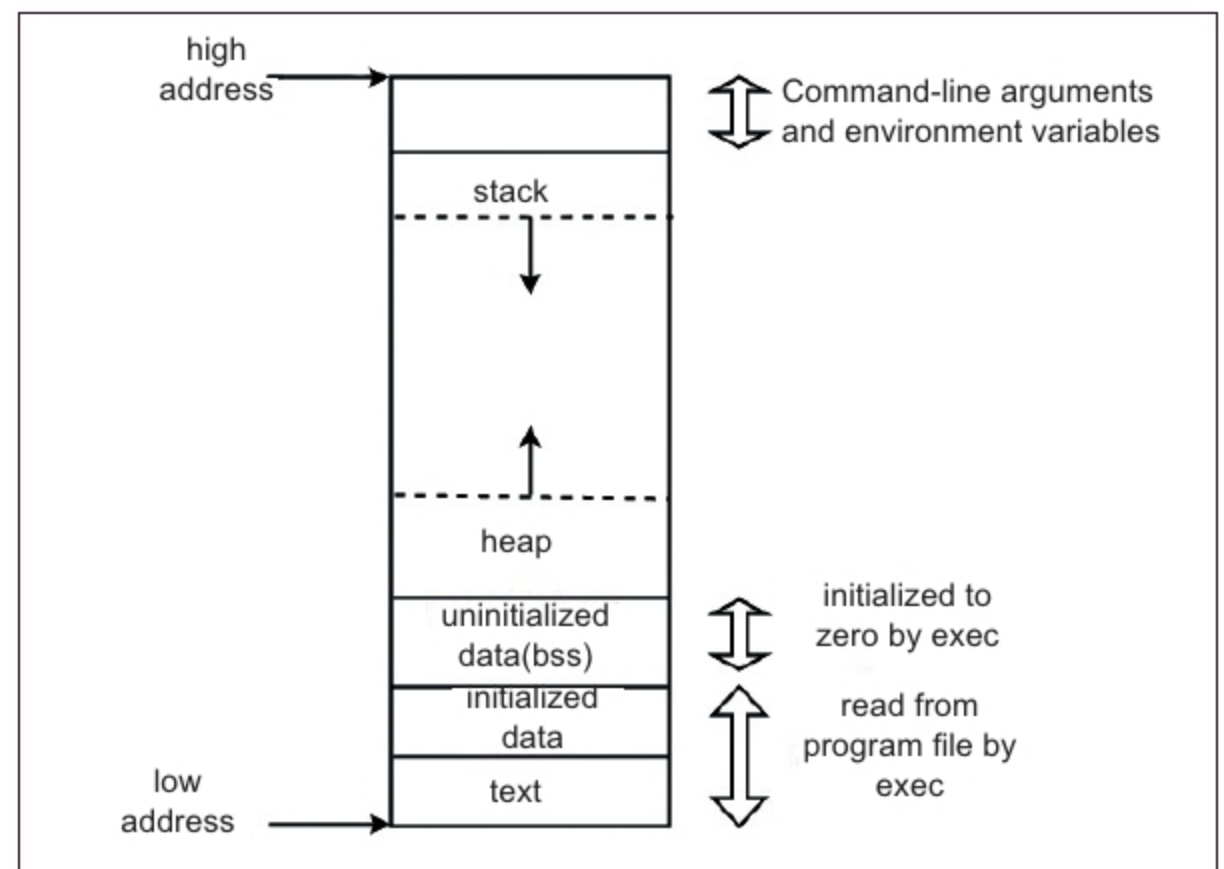


Figure 2: Memory layout of the C program.

instruction, which is to be executed after the function call is over. The scope of local variables is limited to the block they are defined in, and their life time exists within that block. The output remains same but non-compliant atomic code is subject to undefined behavior. We can create re-usable C++ objects and its member functions.

Typically, the compile time allocated memory [*char *str = "OpenSourceForYou"*] is stored in the initialised read-only data segment, and the pointer expression '*++(*str)++*' tries to increment the value of the first 'O' by an ASCII value 1 on the

read-only data segment, resulting in a segmentation fault. On the other hand, dynamically or runtime allocated memory is a read-write memory pointer expression '*++(*str)++*' incremented value of the first 'O' by the ASCII value 1, and the result character 'P' is printed. Note the post-incremented operator result effect after the *printf* statement.

Atomic operations

Another interesting feature of the GCC is atomic operations (volatile *sig_atomic_t*) in the signal handler, but before exploring this, let us discuss the